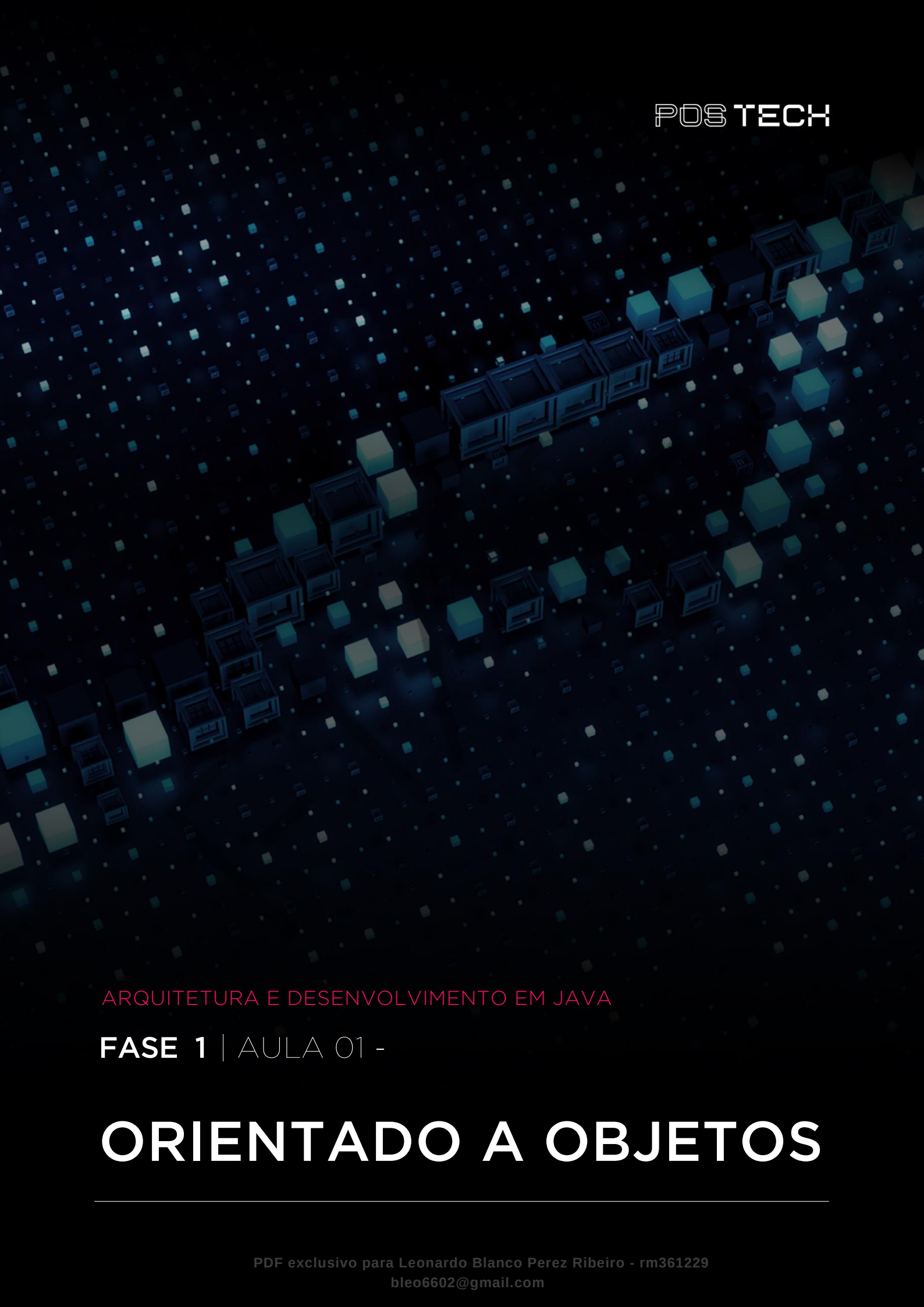




POSTECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 1 | AULA 01 -

ORIENTADO A OBJETOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	50
REFERÊNCIAS.....	51

EMSE

O QUE VEM POR AÍ?

Nesta aula, iremos aprender os conceitos da programação orientada a objetos (POO) e sua aplicabilidade. Teremos exemplos práticos do básico ao avançado, indo além dos casos “comuns” e aplicando esses conceitos em cenários reais.

Abordaremos a diferença entre a programação estruturada e orientada a objetos, permitindo que o aluno e aluna percebam como a POO facilita o desenvolvimento de soluções de software. Com isso, esperamos que possam aplicar de forma correta a POO e extrair todo o potencial desse paradigma de programação fascinante!

HANDS ON

Nas videoaulas, vamos tratar os assuntos e temas essenciais para o entendimento da POO. Os vídeos iniciais são a base para o entendimento e, portanto, são fundamentais para alunos e alunas.

Serão abordados também conceitos mais complexos que permitirão que vocês apliquem corretamente esse conhecimento no desenvolvimento de software.

EMSE

SAIBA MAIS

PROGRAMAÇÃO ESTRUTURADA

É a forma mais simples de programação. Esse paradigma é a base de todos os outros, pois a partir dele aconteceu toda a evolução da programação que temos hoje. Esse tipo enfatiza a linearidade, trazendo legibilidade e clareza na leitura, pelo menos quando se trata de código simples ou pequeno. Não entraremos em detalhes sobre este paradigma, mas o usaremos para comparar com a POO.

No código 1, temos um exemplo de programa estruturado. Em resumo, o programa obtém os dados de alunos informado pelo usuário, faz o cálculo de média das notas e, por fim, salva esses dados (detalhes de carregamento e do salvamento foram suprimidos).

Se notarmos o programa, temos arrays que armazenam dados de alunos (nome, notas, e-mails e idade). Perceba que os dados dos usuários nesses arrays formam uma matriz, em que os dados de cada aluno são indexados pelo índice do array.

As regras de negócio ficam implementadas em funções (ex: "salvaNotas", "validaRegraldade") ou dentro do próprio fluxo do algoritmo (obtendo a média das notas).

O problema surge quando a complexidade do negócio exige uma grande quantidade de regras. Isso implica em grande quantidade de código, funções com alto acoplamento (o que impede o reuso de código) e os arquivos tendem a ficar grandes e difíceis de compreender, com funções espalhadas em vários arquivos.

Essas questões atrapalham a manutenção e as evoluções do sistema e, em alguns casos, torna-se praticamente impossível realizar ajustes sem ocasionar novos erros.

```
public static void salvarGerarMediaDosAlunosNaDisciplina() {  
    final int quantidadeAlunos = obterQuantidadeAlunos();  
    int idDisciplina = obterIdentificadorDisciplina();  
  
    String[] nomesAlunos = new String[quantidadeAlunos];  
    float[] notasAlunos = new float[quantidadeAlunos];  
}
```

```
String[] emailsAlunos = new String[quantidadeAlunos];
int[] idadeAlunos = new int[quantidadeAlunos];

float totalNotasAlunos = 0;
float mediaNotas = 0;

//Obtem os dados dos alunos informado pelo usuário
for (int i = 0; i < quantidadeAlunos; i++) {
    nomesAlunos[i] = obterNomeConsole();
    notasAlunos[i] = obterNotaConsole();
    emailsAlunos[i] = obterEmailConsole();
    idadeAlunos[i] = obterIdadeConsole();
}

//Salva as notas do alunos
salvaNotas(nomesAlunos, notasAlunos, emailsAlunos);

//Aplica uma regra
validaRegraDaIdade(notasAlunos, idadeAlunos);

//Obtem o total
for (int i = 0; i < quantidadeAlunos; i++) {
    totalNotasAlunos += notasAlunos[i];
}

//Obtem a media das notas
mediaNotas = totalNotasAlunos / mediaNotas;

salvarMediaNotas(idDisciplina, mediaNotas);
}
```

Código 1 — Exemplo de Programa Estruturado
Fonte: elaborado pelo autor (2024)

PROGRAMAÇÃO ORIENTADA A OBJETOS

Esse paradigma de programação utiliza o conceito de que tudo no sistema é um objeto. Esses objetos contêm dados (atributos ou propriedades) e operações (ações ou métodos) relacionados ao próprio objeto. Vejamos alguns exemplos.

Se considerarmos um sistema de e-commerce, teremos estes objetos: “Produto”, “Cliente”, “Loja”, “Carrinho de Compra”, “Pedido”, “Pagamento”, “Desconto” etc. Os atributos e operações seriam, por exemplo:

- Carrinho de Compra
 - Atributos:
 - Data Criação.
 - Status.
 - Operações:
 - “Qual o valor total do carrinho?”
 - “Qual o valor total de desconto?”
 - “Qual a data de vigência deste carrinho?”
- Pedido
 - Atributos:
 - Data Criação.
 - Status.
 - Operações:
 - “Qual o valor total do pedido?”
 - “Qual o valor do frete?”

Se considerarmos um sistema de notas de alunos, haveria estes objetos: “Aluno”, “Escola”, “Disciplina”, “Prova” etc. Os atributos e operações seriam, por exemplo:

- Aluno
 - Propriedades:
 - Nome.
 - E-mail.
 - Data de nascimento.

Orientado a Objetos

- Métodos
 - Total de notas.
 - Média das notas.
 - Está aprovado?
- Escola
 - Propriedades:
 - Nome.
 - Endereço.
 - Métodos:
 - Quantidade de alunos.
 - Disciplinas.
 - Média de notas.

Se observarmos atentamente, dependendo do objeto (se for objeto de domínio de negócio), ele pode conter dados cruciais do negócio (propriedades) e operações cruciais de negócio (métodos). Os objetos podem ser trafegados em camadas da aplicação permitindo reuso de código (métodos).

Um objeto (também chamado de instância) é criado a partir de uma “classe”. Classes são trechos de código que definem como um objeto deve ser. Ou seja, na classe são definidos quais atributos e métodos o objeto terá.

No código 2, temos um exemplo de classe que define um aluno. As quatro primeiras linhas são as propriedades de um objeto aluno. Nas demais linhas, temos os métodos que representam operações de regra de negócio.

```
class Aluno {  
    String nome;  
    String email;  
    Integer idade;  
    Double[] notas;
```



```
Double obterTotalNotas() {  
    Double totalNotas = 0D;  
    for (Double nota : notas) {  
        totalNotas += nota;  
    }  
  
    return totalNotas;  
}  
  
boolean isMenorIdade() {  
    return idade < 18;  
}  
}
```

Código 2 — Classe que define um aluno.
Fonte: Elaborado pelo autor (2024)

Assim, em um sistema orientado a objeto, teremos dezenas (ou até mesmo centenas) de classes que definem qualquer tipo de objeto de negócio, visto que, dependendo da linguagem ou plataforma de desenvolvimento, tudo pode ser um objeto, até os códigos internos da própria plataforma de desenvolvimento.

Classes, por si só, não fazem absolutamente nada. Se criarmos em um sistema somente nossas classes, nada ocorrerá, pois estaríamos apenas criando as definições dos objetos. Para que algo aconteça (execução de um fluxo sistêmico), é necessário criarmos os objetos (instâncias) a partir dessas classes. Quando houver instâncias, começamos a operá-las nos fluxos sistêmicos.

Para melhor entendimento, criaremos o mesmo programa do código 1 (programação estruturada), porém usando POO. Neste momento, iremos suprimir alguns detalhes técnicos do código, com o intuito de simplificar e focar no conceito explicado. É importante reforçar que mais adiante passaremos por todos os detalhes e conceitos da POO, então caso se depare com algo novo ou que não entenda de imediato, fique tranquilo(a).

Iniciamos definindo as classes do nosso sistema. Existem técnicas que ajudam nas definições das classes, tipo DDD, que não serão nosso foco aqui. Para nosso exemplo, temos duas classes que representam o negócio:

- Aluno.

Orientado a Objetos

- Disciplina.

No código 3, temos a classe Aluno completamente implementada.

```
class Aluno {  
    String nome;  
    String email;  
    Integer idade;  
    Double[] notas;  
  
    Double obterTotalNotas() {  
        Double totalNotas = 0D;  
        for (Double nota : notas) {  
            totalNotas += nota;  
        }  
  
        return totalNotas;  
    }  
  
    boolean isMenorIdade() {  
        return idade < 18;  
    }  
}
```

Código 3 – Classe Aluno implementada
Fonte: Elaborado pelo autor (2024)

No código 4 temos a classe Disciplina completamente implementada. As primeiras linhas definem as propriedades de “uma disciplina”. Aqui pode haver confusão, pois **não há** uma propriedade que defina o nome de uma disciplina, quando na realidade todas as disciplinas têm um nome. Poderíamos colocar a propriedade “nome” nesta classe, mas não precisamos dessa informação ainda. Além disso, vamos manter esse projeto o mais próximo possível do código original (versão estruturada).

Após as propriedades da classe, temos as implementações das regras de negócio. Note que em ambas as classes, os métodos utilizam os dados contidos da própria classe. Isso não impede esses métodos de receberem qualquer parâmetro

Orientado a Objetos

externo para uso; o mais importante é que os próprios objetos respondem suas ações usando os dados contidos neles mesmos, em suas propriedades.

```
class Disciplina {  
    Integer id;  
    Aluno[] alunos;  
  
    double obterMediaNotas() {  
        return obterTotalNotas() / obterQuantidadeAlunos();  
    }  
  
    void validaAlunosMaiorIdade() {  
        for (Aluno aluno : alunos) {  
            if(aluno.isMenorIdade()) {  
                throw new AlunoMenorIdadeException();  
            }  
        }  
    }  
  
    double obterTotalNotas() {  
        double totalNotas = 0D;  
        for (Aluno aluno : alunos) {  
            totalNotas += aluno.obterTotalNotas();  
        }  
        return totalNotas;  
    }  
  
    int obterQuantidadeAlunos() {  
        return alunos.length;  
    }  
}
```

Código 4 – Classe Disciplina Implementada
Fonte: elaborado pelo autor (2024)

Agora precisamos de um algoritmo que utilize essas classes para enfim chegarmos nos resultados. Temos isso implementado no código 5. O código 5 é igual ao 1, mas foi alterado para utilizar a POO.

Orientado a Objetos

Deixamos diversos comentários nas instruções da classe a fim de ajudar o aluno e aluna em sua compreensão. A seguir, explicamos com detalhes os principais pontos.

```
public static void salvarGerarMediaDosAlunosNaDisciplina() {  
    final int quantidadeAlunos = obterQuantidadeAlunos();  
    int idDisciplina = obterIdentificadorDisciplina();  
  
    //Cria um objeto disciplina sem informações  
    Disciplina disciplina = new Disciplina();  
    //Seta o id da disciplina com o informado pelo usuário  
    disciplina.id = idDisciplina;  
    //Cria um objeto de array de alunos (vazio)  
    disciplina.alunos = new Aluno[quantidadeAlunos];  
  
    //Obtem os dados dos alunos informado pelo usuário  
    for (int i = 0; i < quantidadeAlunos; i++) {  
        //Cria um objeto aluno com dados vazios  
        Aluno aluno = new Aluno();  
        //Cria um array de notas no aluno  
        //com 1 posição e vazio  
        aluno.notas = new Double[1];  
  
        //Seta os dados do objeto aluno  
        //(informado pelo usuário)  
        aluno.nome = obterNomeConsole();  
        aluno.notas[1] = obterNotaConsole();  
        aluno.email = obterEmailConsole();  
        aluno.idade = obterIdadeConsole();  
  
        //Adiciona na lista de alunos  
        //(no objeto disciplina) o aluno criado  
        disciplina.alunos[i] = aluno;  
    }  
  
    //Salva as notas do alunos passando  
    //todo o objeto disciplina (que contem os alunos e sua notas)  
    salvaNotas(disciplina);  
}
```

```
//Aplica uma regra
disciplina.validaAlunosMaiorIdade();

double mediaNotas = disciplina.obterMediaNotas();
salvarMediaNotas(idDisciplina, mediaNotas);
}
```

Código 5 - Método que executa o fluxo sistêmico
Fonte: elaborado pelo autor (2024)

As mudanças mais significativas são os usos dos objetos. Iniciamos criando um objeto disciplina (“new Disciplina()”). Como no algoritmo temos somente uma disciplina, o que faz parte da regra de negócio, criamos somente um objeto deste tipo (Disciplina). O mesmo não acontece com os alunos, pois podemos ter vários alunos, então criamos um objeto aluno dentro do laço “for”. Mas atenção: para cada loop do “for”, criaremos um objeto do tipo aluno.

Voltando para o início do programa, adicionamos as informações da disciplina dentro do objeto criado disciplina. Essas informações são oriundas de um método (suprimido) que obtém os dados que o usuário informa. O mesmo acontece naquelas com o objeto aluno, que foi criado dentro do loop for.

Na última linha do loop for, guardamos o objeto “aluno” na lista de alunos que existe **dentro do objeto disciplina**. Sempre que o fluxo passar nesta linha, o objeto aluno criado no início do for ficara “guardado” nessa lista do objeto aluno em disciplina. Assim, na próxima interação do loop, o novo aluno criado não vai sobrescrever os dados daquele que foi processado no loop anterior.

Ao sair do loop, teremos todos os dados dentro dos objetos (em suas propriedades) e cada objeto contendo comportamentos (métodos) que operam esses dados. Uma regra que antes estava em uma função externa agora está dentro do objeto de negócio. Pode ser que, na primeira análise, o aluno e aluna não perceba o ganho dessa abordagem, mas o maior ganho é que agora os métodos de negócio podem ser acessados de qualquer lugar, basta passar o objeto “disciplina” e chamar seus métodos onde for necessário.

No decorrer do programa, temos uma chamada do método “salvaNotas” (suprimido) que salva os dados em um repositório (banco de dados, arquivo etc.). No código anterior tínhamos um grupo de arrays. Pense como era complexo adicionar novos campos de alunos: teríamos que criar mais arrays para cada novo campo de aluno e alterar vários métodos que antes recebiam menos parâmetros e, agora, precisam receber novos parâmetros. Usando classes e objetos, adicionar novos atributos (propriedades) nas classes fica mais simples e organizado.

Outro ponto importante: suponha que tivéssemos que salvar no banco de dados, além dos dados de aluno (que são as propriedades do objeto aluno), também a média das notas. Perceba que na programação estruturada o cálculo da média poderia estar em dois lugares ou mais, e código repetido configura má prática, ou em uma função compartilhada. Agora, com objeto “disciplina”, esse código está dentro de quem deve saber calcular, basta receber no código que salva os dados (linha 29) o objeto disciplina e acessar o método “obterMediaNotas” deste objeto que ele responderá o valor de acordo com suas propriedades.

Ao final do programa, para efeito didático e de demonstração, fizemos o método “salvarMediaNotas” receber dois parâmetros: o id da disciplina e a média das notas. Mas ele poderia receber somente um parâmetro, que é o objeto “disciplina”, e assim obter as informações acessando suas propriedades e métodos de negócio.

Agora que ficou claro o uso de objetos e conceitos introdutórios, abordaremos com mais profundidade a POO, para que possamos usufruir de todas as possibilidades que esse paradigma nos oferece.

CLASSES

É o código que define a estrutura dos objetos a serem criados. Um sistema pode ter inúmeras classes e normalmente a classe representa um objeto de negócio do sistema ou até mesmo um fluxo de sistema. Já demos algumas classes nos exemplos anteriores, mas o código 6 demonstra uma classe sem quaisquer atributos ou métodos. Não faz sentido ter uma classe sem atributos ou métodos; portanto, no decorrer da aula, evoluiremos essa classe.

```
class Pedido {
```

```
}
```

Código 6 — Classes sem atributos ou métodos
Fonte: elaborado pelo autor (2024)

Classes – atributos

Também chamados de propriedades, são as variáveis que compõem os dados do objeto daquela classe. As propriedades podem ser do tipo primitivo, arrays ou até mesmo de outras classes. Os atributos podem ser vistos no código 7.

Quando uma classe tem em sua propriedade outra classe (ou um array ou lista de outra classe), significa que existe uma relação entre as classes. Isso será explicado mais adiante.

```
class Pedido {  
    LocalDateTime dataHoraCriacao;  
    Double desconto;  
    StatusPedido status;  
    LocalDateTime dataHoraFechamento;  
}
```

Código 7 — Atributos
Fonte: elaborado pelo autor (2024)

Classes – métodos

São as operações que normalmente utilizam as propriedades da classe para efetuar alguma lógica de negócio. No código 8, temos alguns exemplos nas linhas 12 e 17. Note que o método “fecharPedido” não tem retorno, porém ele efetua operações nas propriedades do objeto. Já o “getValorTotalComDesconto” também utiliza as propriedades da classe para efetuar um cálculo de negócio e em seguida retorna o valor calculado.

```
class Pedido {  
    LocalDateTime dataHoraCriacao;  
    Double desconto;
```

```
Produto[] produtos;  
StatusPedido status;  
LocalDateTime dataHoraFechamento;  
  
Double obterValorTotalComDesconto() {  
    Double valorTotal = 0D;  
    for (Produto produto : produtos) {  
        valorTotal += produto.getValor();  
    }  
    return valorTotal - desconto;  
}  
  
void encerrarPedido() {  
    if(status.equals(StatusPedido.PAGO)) {  
        status = StatusPedido.FECHADO;  
        dataHoraFechamento = LocalDateTime.now();  
    } else {  
        throw new StatusPedidoInvalidoException();  
    }  
}
```

Código 8 — Métodos
Fonte: elaborado pelo autor (2024)

Classes – métodos estáticos

Como vimos, uma classe/objeto pode ter propriedades e métodos. Até agora, os métodos apresentados servem para processar alguma coisa, utilizando-se das propriedades do objeto.

Porém, um método pode receber informações (variáveis) por parâmetro e usá-las ou não para operar com as propriedades da classe/objeto.

No código 9, ambos os métodos recebem parâmetros, mas somente o primeiro ("incrementaValorDesconto") utiliza as propriedades do objeto para o cálculo. Por isso o outro método ("somaValores") é um candidato a ser um método estático.

```
class Pedido {  
    LocalDateTime dataHoraCriacao;  
    Double desconto;
```



```
Produto[] produtos;  
StatusPedido status;  
LocalDateTime dataHoraFechamento;  
  
void incrementaValorDesconto(Double valor) {  
    desconto += valor;  
}  
  
Double somaValores(Double valorA, Double valorB) {  
    return valorA + valorB;  
}  
}
```

Código 9 – Método Estático
Fonte: elaborado pelo autor (2024)

O código 10 demonstra como são chamados os métodos da classe do código 9. Observe o código 10 e note que nas duas chamadas dos métodos usamos a instância (objeto) criada anteriormente. Como já aprendemos, essa é a maneira correta de chamar métodos de um objeto.

```
public static void main(String[] args) {  
    Pedido novoPedido = new Pedido();  
    novoPedido.desconto = 20D;  
  
    Double valoreDescontoIncrementar = 15D;  
    novoPedido.incrementaValorDesconto(valoreDescontoIncrementar);  
  
    Double valorA = 10D;  
    Double valorB = 20D;  
    Double resultado = novoPedido.somaValores(valorA, valorB);  
    System.out.println("Resultado: " + resultado);  
}
```

Código 10– Método Estático (2)
Fonte: elaborado pelo autor (2024)

Agora vamos mudar o método “somaValores” para ser estático. O código 11 demonstra essa alteração. Note que a palavra reservada “static” foi adicionada na assinatura do método.

```
class Pedido {  
    LocalDateTime dataHoraCriacao;  
    Double desconto;  
    Produto[] produtos;  
    StatusPedido status;  
    LocalDateTime dataHoraFechamento;  
  
    void incrementaValorDesconto(Double valor) {  
        desconto += valor;  
    }  
  
    static Double somaValores(Double valorA, Double valorB) {  
        return valorA + valorB;  
    }  
}
```

Código 11– Método Estático (3)
Fonte: Elaborado pelo autor (2024)

Assim a chamada desse método não precisa da instância (objeto) da classe. No código 12, comentamos propositalmente os códigos que criam e usam o objeto, para que o aluno e aluna percebam que não é necessário o objeto para chamar métodos estáticos. Então, mudamos o nome da variável do objeto para o nome da classe e atestamos que funciona.

```
public static void main(String[] args) {  
    // Pedido novoPedido = new Pedido();  
    // novoPedido.desconto = 20D;  
    //  
    // Double valoreDescontoIncrementar = 15D;  
    // novoPedido.incrementaValorDesconto(valoreDescontoIncrementar);  
}
```

```
Double valorA = 10D;  
Double valorB = 20D;  
Double resultado = Pedido.somaValores(valorA, valorB);  
System.out.println("Resultado: " + resultado);  
}
```

Código 12– Método Estático (4)
Fonte: elaborado pelo autor (2024)

Normalmente os métodos estáticos são usados em classes utilitárias que possuem vários métodos utilitários. Por exemplo: conversão do objeto “Date” (ou “LocalDate”) para “String”.

Classes – construtores

Construtor é a parte da classe que sempre é chamada quando um objeto daquela classe é criado. Normalmente um construtor possui instruções de código que inicializam as propriedades do objeto que está sendo criado.

Em Java e algumas outras linguagens de programação, utiliza-se o comando “new” para criar objetos de uma classe. No código 13 vemos o uso desse comando, em que dois objetos foram criados e referenciados, cada um com sua variável (“novoPedidoA”, “novoPedidoB”).

```
public static void main(String[] args) {  
    Pedido novoPedidoA = new Pedido();  
    Pedido novoPedidoB = new Pedido();  
}
```

Código 13 — Construtores
Fonte: elaborado pelo autor (2024)

Em Java, toda classe tem um construtor: mesmo que ele não esteja presente (implementado) na classe, haverá um oculto.

Vejamos alguns exemplos. No código 14 temos a definição de uma classe “Pedido”. Nela, temos as declarações de suas propriedades, seu construtor e seus métodos. Esse construtor sem parâmetros é o que chamamos de “construtor default”.

Orientado a Objetos

Ele não é obrigado a existir na classe, pois ficará oculto (caso não seja declarado). Se existir um outro construtor com parâmetros, ele não existirá por default.

Um construtor default ou sem implementação implica que os objetos criados por ele terão suas propriedades nulas (sem valor), visto que elas não foram inicializadas no construtor. Então, após criar um objeto com esse tipo de construtor, é necessário adicionar informações a suas propriedades para que seus métodos possam ser operados corretamente.

```
class Pedido {  
    LocalDateTime dataHoraCriacao;  
    Double desconto;  
    StatusPedido status;  
    LocalDateTime dataHoraFechamento;  
  
    Pedido(){  
    }  
  
    Double obterValorTotalComDesconto() {  
        Double valorTotal = 0D;  
        for (Produto produto : produtos) {  
            valorTotal += produto.getValor();  
        }  
        return valorTotal - desconto;  
    }  
  
    void encerrarPedido() {  
        if(status.equals(StatusPedido.PAGO)) {  
            status = StatusPedido.FECHADO;  
            dataHoraFechamento = LocalDateTime.now();  
        } else {  
            throw new StatusPedidoInvalidoException();  
        }  
    }  
  
    LocalDateTime getDataHoraCriacao() {  
        return dataHoraCriacao;  
    }  
  
    StatusPedido getStatus() {
```

```
        return status;
    }

    LocalDateTime getDataHoraFechamento() {
        return dataHoraFechamento;
    }
}
```

Código 14 — Definição de uma classe “Pedido”
Fonte: elaborado pelo autor (2024)

Uma classe pode ter definidos quantos construtores forem necessários. No código 15, adicionamos três construtores na classe “pedido”. Cada um tem seu grupo de parâmetros e inicializa as variáveis do objeto da maneira mais adequada à necessidade de quem precisa criar e usar este objeto.

Temos o uso de palavra reservada “this”. Ela é necessária para diferenciar a propriedade da classe que tem mesmo nome da recebida por parâmetro.

```
class Pedido {
    LocalDateTime dataHoraCriacao;
    Double desconto;
    Produto[] produtos;
    StatusPedido status;
    LocalDateTime dataHoraFechamento;

    Pedido(Double desconto, Produto[] produtos){
        dataHoraCriacao = LocalDateTime.now();
        this.desconto = desconto;
        status = StatusPedido.AGUARDANDO_PAGAMENTO;
        this.produtos = produtos;
    }

    Pedido(Produto[] produtos){
        dataHoraCriacao = LocalDateTime.now();
        status = StatusPedido.AGUARDANDO_PAGAMENTO;
        this.produtos = produtos;
    }

    Pedido(){
        dataHoraCriacao = LocalDateTime.now();
    }
}
```

```
}
```

```
Double obterValorTotalComDesconto() {  
    Double valorTotal = 0D;  
    for (Produto produto : produtos) {  
        valorTotal += produto.getValor();  
    }  
  
    return valorTotal - desconto;  
}  
  
void encerrarPedido() {  
    if(status.equals(StatusPedido.PAGO)) {  
        status = StatusPedido.FECHADO;  
        dataHoraFechamento = LocalDateTime.now();  
    } else {  
        throw new StatusPedidoInvalidoException();  
    }  
}  
  
LocalDateTime getDataHoraCriacao() {  
    return dataHoraCriacao;  
}  
  
StatusPedido getStatus() {  
    return status;  
}  
  
LocalDateTime getDataHoraFechamento() {  
    return dataHoraFechamento;  
}  
}
```

Código 15 — Adicionamos três construtores na classe “pedido”
Fonte: elaborado pelo autor (2024)

O código 16 demonstra a criação de objetos com diferentes construtores, que foram definidos na classe do código anterior.

```
public static void main(String[] args) {  
    Produto produtos[] = new Produto[10];  
  
    Pedido novoPedidoA = new Pedido();  
    Pedido novoPedidoB = new Pedido(produtos);  
    Pedido novoPedidoC = new Pedido(25D, produtos);  
}
```

Código 16 — criação de objetos com diferentes construtores
Fonte: elaborado pelo autor (2024)

MODIFICADORES DE ACESSO

Nos exemplos até aqui, conseguimos mudar diretamente os dados (atributos) dos objetos. Isso não é recomendado, pois dessa forma podemos mudar a integridade do objeto, o que pode ocasionar diversos erros ou bugs.

Proteger o acesso aos dados dos objetos é um dos princípios fundamentais da POO. Este princípio se chama “Encapsulamento” e o veremos com mais detalhes adiante. Por ora, precisamos nos preocupar em proteger nossos objetos de acesso externo.

O uso de modificadores de acesso é um padrão que se estende a diversas linguagens de programação, não sendo exclusiva do Java. Aqui focaremos nos exemplos em Java.

Os níveis de proteção vão de “classe”, “pacote” e “subclasse” até “externo”.

- Classe: somente podem ser acessados diretamente de dentro da própria classe.
- Pacote: podem ser acessados diretamente de dentro da própria classe e de classes de mesmo pacote.
- Subclasse: podem ser acessados de classes filhas.
- Externo: acessos diretamente de qualquer outro pacote ou parte do sistema.

As palavras reservadas utilizadas para proteger atributos ou métodos são:

- `public`
 - Classe
 - Pacote
 - Subclasse
 - Externo
- `protected`
 - Classe
 - Pacote
 - Subclasse
- `default`
 - Classe
 - Pacote
- `private`
 - Classe

Vamos aos nossos exemplos. No código 17, as propriedades da classe foram protegidas de qualquer acesso externo, pois utilizam “private” como modificador de acesso, assim como o método “obtemTotalTaxas”, que é utilizado internamente na classe, mais precisamente no método “getValorFinal”. Além disso, temos métodos públicos que podem ser acessados de qualquer parte do sistema.

```
public class Produto {  
    private Double valor;  
    private boolean origemNacional;  
    private Double[] taxas;  
  
    public Produto{
```



```
        Double valor,  
        boolean origemNacional,  
        Double[] taxas) {  
    super();  
    this.valor = valor;  
    this.origemNacional = origemNacional;  
    this.taxas = taxas;  
}  
  
    public Double getValor() {  
        return valor;  
    }  
  
    public boolean isOrigemNacional() {  
        return origemNacional;  
    }  
  
    public Double[] getTaxas() {  
        return taxas;  
    }  
  
    public Double getValorFinal() {  
        return valor + obtemTotalTaxas();  
    }  
  
    private Double obtemTotalTaxas() {  
        Double totalTaxas = 0D;  
        for (Double taxa : taxas) {  
            totalTaxas += taxa;  
        }  
    }  
}
```

```
}  
  
return totalTaxas;  
  
}
```

Código 17 — Propriedades da classe
Fonte: elaborado pelo autor (2024)

INTERFACES

Não devemos confundir o termo “interface” aqui citado com interfaces de tela ou sistema de front-end. Na POO, interfaces são elementos que servem para definir “contratos” ou “padrões de comunicação” que podem e devem ser utilizados nas integrações entre classes de diferentes camadas de um sistema.

Diferente de classes, as interfaces somente definem a assinatura (estrutura) dos métodos, mas não definem a implementação. As implementações desses métodos ficam a cargo das classes que implementam a interfaces.

No código 18 temos um exemplo de interface. Note que uma interface apenas define os métodos, mas sem implementá-los.

```
public interface PedidoDatabaseGateway {  
    void salvar(Pedido pedido);  
    Pedido[] obterTodos();  
}
```

Código 18 – Exemplo de interface
Fonte: Elaborado pelo autor (2024)

Vejamos um exemplo prático de uso. Imagine que temos um sistema que precisa salvar dados de pedido em um banco de dados. Porém, por uma questão de regra fictícia no nosso exemplo, ora o banco de dados deve ser MySQL, ora Postgree. Nosso objetivo é conseguir usar qualquer um dos bancos de dados sem precisar alterar a camada de regra de negócio. Para conseguir isso, faremos uso de interface.

No código 19 temos o código de nossa aplicação ainda em desenvolvimento. Note que há o condicional de salvar no MySQL ou Postgree. Nesta parte, o aluno e aluna podem pensar em já colocar dentro (“if e “else”) o código de acesso ao database.

Apesar de funcionar, isso não é recomendado, pois o correto é separar o acesso ao banco de dados da camada de negócio.

```
public class App {  
    public static void main(String[] args) {  
        Pedido pedido =  
            obterPedidoInformadoPeloUsuario();  
        if(pedido.contemProdutoInternacional()){  
            //TODO: Deve salvar no mySql  
        } else {  
            //TODO: Deve salvar no postgree  
        }  
    }  
}
```

Código 19 — Aplicação em desenvolvimento
Fonte: elaborado pelo autor (2024)

Uma vez definida a interface (código 18), criamos as respectivas **classes** de implementações (códigos 20 e 21).

```
public class PedidoMySQLGateway implements  
    PedidoDatabaseGateway {  
    @Override  
    public void salvar(Pedido pedido) {  
        // código de mysql  
    }  
    @Override  
    public Pedido[] obterTodos() {  
        // código de mysql  
        return null;  
    }  
}
```

Código 20 - Implementação da interface para MySQL
Fonte: elaborado pelo autor (2024)

```
public class PedidoPostgreeGateway implements
                                   PedidoDatabaseGateway {

    @Override
    public void salvar(Pedido pedido) {
        // código de postgree
    }

    @Override
    public Pedido[] obterTodos() {
        // código de postgree
        return null;
    }
}
```

Figura 21 - Implementação da interface para Postgree
Fonte: elaborado pelo autor (2024)

Havendo as classes de implementação da interface, podemos utilizá-las por meio desta para resolver nosso problema (código 19).

Observe o código 22. Iniciamos obtendo o pedido com método “obtemPedidoInformadoPeloUsuario”. Em seguida criamos uma variável do **tipo da interface** (não das classes). Essa variável do tipo da interface pode receber qualquer **objeto que a implemente**, que no caso são as classes “PedidoMySQLGateway” e “PedidoPostgreeGateway”. Dentro do if/else associamos a variável da interface à instância de um ou outro objeto das classes que a implementam. Por fim, usamos essa variável para salvar o pedido.

```
public static void main(String[] args) {
    Pedido pedido = obterPedidoInformadoPeloUsuario();
    PedidoDatabaseGateway pedidoDatabaseGateway;
    if(pedido.contemProdutoInternacional()) {
        pedidoDatabaseGateway = new PedidoMySQLGateway();
    } else {
        pedidoDatabaseGateway = new
                                PedidoPostgreeGateway();
    }
}
```

```
pedidoDatabaseGateway.salvar(pedido);  
}
```

Código 22 — Variável do tipo da interface
Fonte: elaborado pelo autor (2024)

Note que o código de “salvar” não é de conhecimento de nosso programa. Se no futuro surgir a necessidade de outro database, basta criar a classe e adicionar a criação da instância no condicional, assim o save não precisará mudar.

O uso de interfaces permite o uso de SOLID e a aplicação de Padrões de Projetos, deixando o código limpo e com grande flexibilidade de evoluções.

ENCAPSULAMENTO

Como explicado anteriormente, um objeto deve manter seus dados (atributos) protegidos do meio externo. Um objeto muitas vezes é criado tendo seus dados preenchidos pelo construtor de sua classe e ele pode ser trafegado por diversas camadas do sistema. Se alguma dessas camadas, de forma equivocada (bug), alterar diretamente umas dessas propriedades, todos os métodos de negócio do objeto passam a ter resultados diferentes do que deveriam ter.

Por exemplo, imagine nossa classe Pedido (código 23). Note que suas propriedades estão com modificadores de acesso público.

```
public class Pedido {  
    public Double desconto;  
    public Produto[] produtos;  
  
    public Double obterValorTotalComDesconto() {  
        Double valorTotal = 0D;  
        for (Produto produto : produtos) {  
            valorTotal += produto.getValor();  
        }  
  
        return valorTotal - desconto;  
    }  
}
```

Código 23 — Modificadores de acesso público
Fonte: elaborado pelo autor (2024)

Suponha que no início do fluxo sistêmico criamos um objeto pedido que deve ter desconto igual a 5 e esse valor de desconto deve ser mantido até o fim. No início do fluxo passamos a usar este objeto (na verdade seus métodos de regras de negócio) que se utiliza do valor de desconto que indicamos na criação do objeto. O sistema utiliza os métodos desse objeto para calcular valores, para salvá-los e ao tomar decisões.

Agora, imagine que no meio do fluxo sistêmico, um desenvolvedor equivocadamente muda o valor do desconto deste objeto de 5 para 50. Sim, é um bug, mas desse ponto em diante os métodos deste objeto passam a ter outro comportamento e outros resultados. Assim, uma parte do fluxo seguiu usando o objeto com seus métodos operando com desconto igual 5 e a outra metade passou a usar os métodos operando com valor igual a 50.

Para evitarmos isso, precisamos fechar o acesso a esses atributos. Dependendo do caso, alguns métodos também não devem ser acessados de qualquer lugar. Isso evita propagação de bugs, já que desta forma mantemos a integridade do objeto.

Ao usar essa abordagem estamos utilizando um dos principais conceitos da POO, que se chama “Encapsulamento”. O código 24 traz a classe pedido da forma correta. Note que as propriedades da classe estão fechadas com `private` e temos um construtor que cria o objeto com os dados recebidos e dados default de um novo objeto pedido. A seguir, temos métodos de negócio e métodos “get” que permitem acesso indireto a propriedades do objeto para leitura.

```
public class Pedido {  
    private LocalDateTime dataHoraCriacao;  
    private Double desconto;  
    private Produto[] produtos;  
    private StatusPedido status;  
    private LocalDateTime dataHoraFechamento;  
  
    public Pedido(Double desconto, Produto[] produtos){  
        dataHoraCriacao = LocalDateTime.now();  
    }  
}
```

```
this.desconto = desconto;
status = StatusPedido.AGUARDANDO_PAGAMENTO;
this.produtos = produtos;
}

public Pedido(Produto[] produtos){
    dataHoraCriacao = LocalDateTime.now();
    status = StatusPedido.AGUARDANDO_PAGAMENTO;
    this.produtos = produtos;
}

public Pedido(){
    dataHoraCriacao = LocalDateTime.now();
}

public Double obterValorTotalComDesconto(){
    Double valorTotal = 0D;
    for (Produto produto : produtos) {
        valorTotal += produto.getValor();
    }

    return valorTotal - desconto;
}

public void encerrarPedido(){
    if(status.equals(StatusPedido.PAGO)) {
        status = StatusPedido.FECHADO;
        dataHoraFechamento = LocalDateTime.now();
    } else {
        throw new StatusPedidoInvalidoException();
    }
}

public LocalDateTime getDataHoraCriacao(){
    return dataHoraCriacao;
}

public StatusPedido getStatus(){
    return status;
}
```

```
}  
  
public LocalDateTime getDataHoraFechamento() {  
    return dataHoraFechamento;  
}  
}
```

Código 24 — Classe pedido da forma correta
Fonte: elaborado pelo autor (2024)

Um objeto criado com uma classe dessa forma terá sua integridade mantida do início ao fim do seu ciclo de vida. Ou seja, ele será criado em uma camada do sistema e poderá percorrer várias outras sem que suas propriedades sejam alteradas equivocadamente.

Importante: uma prática **errada** que se tem adotado no mercado é o uso inadequado de “get” e “set”. Métodos “get” e “set” são os métodos utilizados para acesso **indireto** a propriedades do objeto. O código 25 demonstra os métodos “get” e “set” de uma classe Produto. Neste exemplo, esta classe está **ferindo o conceito de encapsulamento**, pois permite o acesso indireto a todos os seus dados.

```
public class Produto {  
    private Double valor;  
    private boolean origemNacional;  
    private Double[] taxas;  
    public Double getValor() {  
        return valor;  
    }  
    public void setValor(Double valor) {  
        this.valor = valor;  
    }  
    public boolean isOrigemNacional() {  
        return origemNacional;  
    }  
    public void setOrigemNacional(boolean origemNacional) {  
        this.origemNacional = origemNacional;  
    }  
    public Double[] getTaxas() {  
        return taxas;  
    }  
}
```



```
public void setTaxas(Double[] taxas) {  
    this.taxas = taxas;  
}  
}
```

Código 25 — Ferindo o conceito de encapsulamento
Fonte: elaborado pelo autor (2024)

É importante ressaltar que algumas vezes é necessário esse tipo de método, já que de alguma forma precisamos acessar os dados do objeto. Então ter acesso via get ou set, de forma pontual e controlada, é aceitável, porém deve ser evitado sempre que possível.

RELACIONAMENTO ENTRE CLASSES E OBJETOS

Trata-se das relações entre os objetos das classes. Um objeto pode estar relacionado somente a um ou a uma lista de outros objetos, de outras classes ou da própria classe. A seguir, veremos os tipos de relacionamentos e suas direções.

Por exemplo, um objeto da classe Professor pode ter relação com um ou vários objetos da classe Aluno, assim como um aluno pode ter relação com um ou vários objetos da classe professores. Essa relação é denominada “muitos para muitos”.

Por exemplo: um carro tem um motor e um motor é de um carro. Uma pessoa pode ter vários celulares, mas um celular é somente de uma pessoa. Uma pessoa pode ter vários pets, mas um pet é de uma pessoa. Um motorista pode dirigir nenhum ou vários ônibus e um ônibus pode ter sido dirigido por nenhum ou vários motoristas.

Os relacionamentos variam de acordo com as regras de negócio de um sistema. Por mais que na realidade um pet possa pertencer a uma ou mais pessoas, pode ser que, para um dado sistema, pertencer somente a uma pessoa é o que faz sentido.

Falando em código, os relacionamentos são criados colocando essa relação no atributo das classes. O código 26 apresenta um exemplo em que um professor ensina nenhum ou mais alunos (note a segunda propriedade desta classe Professor).

```
public class Professor {  
    private String nome;  
    private List<Aluno> alunos;  
    public Professor(String nome, List<Aluno> alunos) {  
        super();  
        this.nome = nome;  
        this.alunos = alunos;  
    }  
    public String getNome() {  
        return nome;  
    }  
    public List<Aluno> getAlunos() {  
        return alunos;  
    }  
}
```

Código 26 — Exemplo onde um professor ensina nenhum ou mais alunos
Fonte: elaborado pelo autor (2024)

Naturalmente, uma classe pode se relacionar com várias outras e assim adicionaremos as relações nas propriedades classes.

Relacionamento entre classes/objetos — Associação

A “Associação” é o tipo de relacionamento mais simples entre objetos. Ela indica que um objeto se relaciona com outro. Exemplo: um professor ensina vários alunos, já um aluno é instruído por um professor. Veja o código desse exemplo a seguir (código 27 e 28).

```
public class Professor {  
    private String nome;  
    private List<Aluno> alunos;  
    public Professor(String nome, List<Aluno> alunos) {  
        super();  
        this.nome = nome;  
        this.alunos = alunos;  
    }  
}
```

```
}  
  
public String getNome() {  
    return nome;  
}  
  
public List<Aluno> getAlunos() {  
    return alunos;  
}  
  
}
```

Código 27 - Relação de um Professor com vários alunos
Fonte: elaborado pelo autor (2024)

```
class Aluno {  
    private String nome;  
    private String email;  
    private Integer idade;  
    private Double[] notas;  
    private Professor professor;  
  
    public Double obterTotalNotas() {  
        Double totalNotas = 0D;  
        for (Double nota : notas) {  
            totalNotas += nota;  
        }  
  
        return totalNotas;  
    }  
  
    public boolean isMenorIdade() {  
        return idade < 18;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
}
```

```
public String getEmail() {  
    return email;  
}  
  
public Integer getIdade() {  
    return idade;  
}  
  
public Double[] getNotas() {  
    return notas;  
}  
}
```

Código 28 - Relação de aluno com um professor
Fonte: elaborado pelo autor (2024)

É importante reforçar que um relacionamento pode ser unidirecional ou bidirecional. Nesse exemplo (código 27 e 28) é um relacionamento bidirecional, já que ambas as classes mapeiam as relações (contêm em suas propriedades a outra classe). Caso fosse unidirecional, somente uma das classes teria a relação.

Isso é utilizado a depender do contexto de negócio. Por exemplo, pode ser importante um professor saber dos alunos para determinadas operações de negócio (métodos), mas o contrário pode não ser necessário. Teríamos então professor com a lista de alunos em sua propriedade, mas aluno não teria professor em sua propriedade.

Relacionamento entre classes/objetos — Agregação

É um relacionamento especial que representa todo/parte. Aqui, um objeto é mais fraco e perde o sentido de sua existência se a outra parte não existir. Por exemplo: uma biblioteca possui livros. Os livros podem existir sem a biblioteca, já uma biblioteca sem livros perde o sentido.

Outro exemplo: uma casa tem espelhos, mas ela não precisa de espelhos para existir. Esse relacionamento, apesar de representar o conceito de domínio de negócio,

não muda no código: a relação acontece definindo a classe relacionada em sua propriedade.

Relacionamento entre classes/objetos — Composição

É um tipo mais forte de uma relação entre classes e objetos. Os objetos relacionados representam um todo. Se o todo deixar de existir, as partes também deixam de existir.

Exemplo: um carro tem um motor. Se o carro deixar de existir, o motor também deixa de existir. Outro exemplo: uma casa precisa de telhado para existir.

No código da classe, nada vai mudar em relação ao que já falamos anteriormente.

Relacionamento entre classes/objetos — Herança

É o relacionamento que representa um “é um” entre uma classe base (superclasse) e uma classe derivada (subclasse). A subclasse pode herdar os atributos e métodos da superclasse se o modificador de acesso for diferente de “private”.

Na prática, é uma extensão de outra classe, já que um objeto pode ter todos os atributos e métodos de outra classe fora os seus próprios atributos e métodos. Pode ainda mudar o comportamento de métodos herdados (sobrescrita de método). Vamos trazer alguns exemplos.

Inicialmente veremos um caso **sem** aplicar herança. Analisamos os problemas e depois alteramos para uso de herança, vendo as diferenças e benefícios do uso de herança.

Suponha uma classe Veículo com suas propriedades e métodos. Veículo é muito genérico e representa qualquer tipo de veículo. O código 29 demonstra essa classe. Suponha que os métodos executem instruções de sistema necessárias para o funcionamento de qualquer tipo de veículo, ou seja, elas sempre precisam ser executadas para qualquer tipo de veículo.

```
public class Veiculo {
```

```
protected String placa;  
protected String modelo;  
protected String cor;  
  
protected void buzinar() {  
    System.out.println("Emite som de buzina");  
}  
  
protected void abastecer() {  
    System.out.println("Executa passos  
    necessários de abastecimento de qualquer veículo");  
}  
  
protected void movimentar() {  
    System.out.println("Executa passos  
    necessários de movimento de qualquer veículo");  
}
```

Código 29 — Classe Veículo
Fonte: elaborado pelo autor (2024)

Agora precisamos de dois tipos de veículos mais específicos: um que ande na água e outro em terra. Nossa classe veículo não está preparada para essas especializações. Neste ponto, devemos tomar algumas decisões pensando em boas práticas de programação, a fim de permitir o melhor reuso de código e evolução do sistema.

Tecnicamente, existem diversas possibilidades de implementação, mas precisamos saber usar a correta. As outras, apesar de funcionarem, trarão problemas de falta de reuso de código, de manutenção e de evolução do sistema. Iremos expor algumas delas aqui (sem uso de herança, por enquanto), pois são falhas comuns que pessoas desenvolvedoras cometem. É importante sabermos as formas erradas para evitá-las.

A primeira solução **equivocada** e a mais comum seria colocar na classe Veículo uma propriedade “tipo de veículo”. Assim, ao criar uma instância dessa classe, recebemos pelo construtor qual o tipo de veículo. Nos métodos da classe Veículo, colocamos condições (“if”) para executar o código correspondente ao tipo de veículo.

Apesar de funcionar, essa é uma das piores soluções, pois se a quantidade de tipos de veículos aumentar, teremos que fazer mais condicionais e isso tende a crescer “infinitamente”. Sem contar que nossa classe estaria ferindo alguns dos princípios SOLID, visto que ela teria mais de uma responsabilidade, por exemplo, com a implementação de mais de um veículo na mesma classe.

Outra solução **equivocada**, ainda sem usar herança, seria criar duas classes especializadas que **não herde** de veículo e cada classe teria seu código específico de carro e barco. Mas elas, ainda assim, devem executar as instruções da classe Veículo, pois conforme mencionamos essas instruções precisam ser executadas **para todo tipo de veículo**. Vejamos os códigos 30 e 31, que representam nossas novas classes ainda de forma errada.

```
public class CarroErrado {  
    protected String placa;  
    protected String modelo;  
    protected String cor;  
  
    protected void buzinar() {  
        System.out.println("Emite som de buzina");  
    }  
  
    protected void abastecer() {  
        System.out.println("Executa passos necessários  
de abastecimento de qualquer veículo");  
        System.out.println("Executa passos necessários  
para abastecimento de carro");  
    }  
  
    protected void movimentar() {  
        System.out.println("Executa passos necessários  
de movimento de qualquer veículo");  
        System.out.println("Executa passos necessários  
de movimento de carro");  
    }  
}
```

Código 30 – Classe de Carro de forma errada
Fonte: elaborado pelo autor (2024)

```
public class BarcoErrado {  
    protected String modelo;  
    protected String cor;  
    protected String tipoLeme;  
  
    protected void buzinar() {  
        System.out.println("Emite som de buzina");  
    }  
  
    protected void abastecer() {  
        System.out.println("Executa passos necessários  
de abastecimento de qualquer veículo");  
        System.out.println("Executa passos necessários  
para abastecimento de barco");  
    }  
  
    protected void movimentar() {  
        System.out.println("Executa passos necessários  
de movimento de qualquer veículo");  
        System.out.println("Executa passos necessários  
de movimento de barco");  
    }  
}
```

Código 31 – Classe de Barco de forma errada
Fonte: elaborado pelo autor (2024)

Analisando o código dessas classes, percebemos que cada uma tem suas propriedades específicas, afinal as propriedades de barco são diferentes das de carro e vice-versa. No entanto algumas propriedades são comuns aos dois e por isso elas se repetiram nas duas classes (cor e modelo). Esse é o primeiro problema.

Observando os métodos, percebemos que as instruções comuns a qualquer veículo estão repetidas nas três classes (Veículo, Carro e Barco) e o método “buzinar”, que é o mesmo para todos os veículos, se repete em todas as classes. Esse é o segundo problema.

Se o aluno ou aluna ainda não percebeu a dimensão do problema, veremos mais alguns cenários. Nos exemplos demonstrados, as instruções comuns às classes são somente as linhas “System.out.println”, mas em um caso real poderiam ser dezenas ou centenas de instruções de linhas de código. Se pensarmos que poderia haver dezenas de classes especializada de veículos, imagine precisar mudar uma regra comum? Teríamos que mudar em dezenas de classes.

Agora, vamos mudar esta última solução para a correta usando herança. Basicamente ajustaremos as classes filhas (Carro e Barco) para estender a classe base Veículo.

```
public class Carro extends Veiculo {
    protected String placa;

    @Override
    protected void abastecer() {
        super.abastecer();
        System.out.println("Executa passos necessários
        de abastecimento de carro");
    }

    @Override
    protected void movimentar() {
        super.movimentar();
        System.out.println("Executa passos necessários
        de movimento de carro");
    }
}
```

Código 32 – Classe Carro correta
Fonte: elaborado pelo autor (2024)

```
public class Barco extends Veiculo {
    protected String tipoLeme;

    @Override
    protected void abastecer() {
        super.abastecer();
        System.out.println("Executa passos necessários
        de abastecimento de barco");
    }
}
```

```
@Override
protected void movimentar() {
    super.movimentar();
    System.out.println("Executa passos necessários
de movimento de barco");
}
}
```

Código 33 – Classe Barco correta
Fonte: Elaborado pelo autor (2024)

Vejamos os códigos 32 e 33. Em ambas as classes adicionamos a palavra reservada do Java “extends” na declaração da classe. Também ajustamos suas propriedades deixando somente as específicas de cada classe. Nos métodos, utilizamos a palavra reservada “super” para que seja chamado o método da superclasse. Essa chamada “super” não é obrigatória, mas para nosso exemplo ela é necessária para executar códigos de qualquer veículo. Quanto ao método “buzinar” ele não está definido nessas classes, mas os objetos delas podem utilizar, pois elas herdaram da superclasse. Em Java existe a anotação “Override”, que é opcional, mas sugerida quando alteramos o comportamento de métodos da classe base.

Essa técnica de escrever o mesmo método da superclasse nas classes filhas chama-se “**Sobrescrita**”. É um recurso muito poderoso da orientação a objetos. A sobrescrita deve sempre ser utilizada quando precisamos alterar o comportamento de métodos existentes da superclasse.

Com isso, vimos que o uso de herança possibilitou o reuso de código e trouxe a **coesão**. Esse termo na POO significa que uma classe tem propósito e responsabilidade bem definida. Em breve, veremos como a POO permite um código limpo e desacoplado.

O código 34 demonstra o uso das classes anteriores em um programa.

```
public static void main(String[] args) {
    System.out.println("-- VEICULO --");
    Veiculo veiculo = new Veiculo();
    veiculo.buzinar();
    veiculo.movimentar();
}
```

```
veiculo.abastecer();

System.out.println("-- CARRO --");
Carro carro = new Carro();
carro.buzinar();
carro.movimentar();
carro.abastecer();

System.out.println("-- BARCO --");
Barco barco = new Barco();
barco.buzinar();
barco.movimentar();
barco.abastecer();
}
```

Código 34 – Programa usando as classes com herança
Fonte: elaborado pelo autor (2024)

```
-- VEICULO --
Emite som de buzina
Executa passos necessários de movimento de qualquer veículo
Executa passos necessários de abastecimento de qualquer veículo

-- CARRO --
Emite som de buzina
Executa passos necessários de movimento de qualquer veículo
Executa passos necessários de movimento de carro
Executa passos necessários de abastecimento de qualquer veículo
Executa passos necessários de abastecimento de carro

-- BARCO --
Emite som de buzina
Executa passos necessários de movimento de qualquer veículo
Executa passos necessários de movimento de barco
Executa passos necessários de abastecimento de qualquer veículo
Executa passos necessários de abastecimento de barco
```

Código 35 – Console da Aplicação
Fonte: elaborado pelo autor (2024)

CLASSES ABSTRATAS

Classes abstratas são um tipo especial de classes do qual não é possível criar instâncias. Se tentarmos criar instâncias de objetos de classe abstrata teremos um erro de compilação. Pode parecer estranho, mas ela tem um papel importante na POO.

No nosso exemplo de veículos, a classe Veículo é muito genérica. Não faz sentido criarmos objetos dela já que precisamos que classes mais específicas (Carro, Moto, Bicicleta etc.) implementem **como** suas ações funcionam. “Veículo” não sabe (ou não deveria saber) como uma moto “liga” e como um carro “liga”. Precisamos deixar que as subclasses implementem esses métodos.

Então em uma classe abstrata, podemos definir métodos genéricos para uso nas subclasses e assinaturas de métodos que obrigam que subclasses as implementem, tendo um comportamento parecido, mas não igual, a interfaces (que vimos anteriormente).

Vamos alterar nossa classe veículo para que seja abstrata. Os códigos 36, 37 e 38 demonstram isso.

```
public abstract class Veiculo {  
    protected String modelo;  
    protected String cor;  
  
    protected void buzinar() {  
        System.out.println("Emite som de buzina");  
    }  
  
    protected void abastecer() {  
        System.out.println("Executa passos necessários  
de abastecimento de qualquer veículo");  
    }  
  
    protected void movimentar() {  
        System.out.println("Executa passos necessários  
de movimento de qualquer veículo");  
    }  
}
```

```
protected abstract void ligar();  
  
}
```

Código 36 — Alterando classe veículo para que seja abstrata (1)
Fonte: elaborado pelo autor (2024)

No código 36, adicionamos a palavra reservada “abstract” na declaração da classe. Também adicionamos um novo método no final da classe. Esse método é um método abstrato e não permite implementação, mas as classes filhas são obrigadas a implementá-los; caso contrário, haverá erro de compilação.

No código 37 e 38 temos as duas classes filhas de Veículo. As duas implementam o método abstrato “liga”, sendo que cada uma delas sabe como fazer isso. Já os demais métodos continuam acessando o código comum da classe base (usando “super”) e executando seus próprios códigos.

```
public class Carro extends Veiculo {  
    protected String placa;  
  
    @Override  
    protected void abastecer() {  
        super.abastecer();  
        System.out.println("Executa passos necessários  
de abastecimento de carro");  
    }  
  
    @Override  
    protected void movimentar() {  
        super.movimentar();  
        System.out.println("Executa passos necessários  
de movimento de carro");  
    }  
  
    @Override  
    protected void ligar() {  
        System.out.println("Executa passos necessários de  
ligar carro");  
    }  
}
```

Código 37 — Alterando classe veículo para que seja abstrata (2)
Fonte: elaborado pelo autor (2024)

```
public class Barco extends Veiculo {  
    protected String tipoLeme;  
  
    @Override  
    protected void abastecer() {  
        super.abastecer();  
        System.out.println("Executa passos necessários  
de abastecimento de barco");  
    }  
  
    @Override  
    protected void movimentar() {  
        super.movimentar();  
        System.out.println("Executa passos necessários  
de movimento de barco");  
    }  
  
    @Override  
    protected void ligar() {  
        System.out.println("Executa passos necessários de  
ligar barco");  
    }  
}
```

Código 38 — Alterando classe veículo para que seja abstrata (3)
Fonte: elaborado pelo autor (2024)

Classes Abstratas x Interfaces

As interfaces somente definem as assinaturas que suas implementações devem implementar. Elas não têm atributos (mas podem definir constantes) e não têm métodos com implementações. Desde o Java 8, é permitido uso de métodos com implementações nas interfaces, chamados de métodos “default”.

Já as classes abstratas, como vimos, podem ter métodos abstratos, que forçam as classes filhas a implementar, e métodos não abstratos, que podem ser reutilizados pelas classes filhas ou sobrescritos.

Então, quando usar uma ou outra? Normalmente quando percebemos que alguns códigos são comuns nas filhas, utilizamos classes abstratas. Se temos a percepção que não haverá códigos comuns, vamos para a interface.

Por fim, tanto a interface quanto as classes abstratas podem ser utilizadas no próximo assunto e um dos mais importantes da POO: o Polimorfismo.

POLIMORFISMO

Suponhamos um cenário em que temos vários objetos de subclasses de Veículo e precisamos executar todos seus métodos. A maneira direta e **sem** uso de polimorfismo é demonstrada no código 39.

```
public static void main(String[] args) {

    List<Carro> obterCarros = obterCarros();
    List<Barco> obterBarcos = obterBarcos();
    List<Bicicleta> obterBicicletas = obterBicicletas();

    for (Carro carro : obterCarros) {
        carro.movimentar();
    }
    for (Barco barco : obterBarcos) {
        barco.movimentar();
    }
    for (Bicicleta bicicleta : obterBicicletas) {
        bicicleta.movimentar();
    }
}
```

Código 39 — Sem uso de polimorfismo
Fonte: elaborado pelo autor (2024)

Note que iniciamos o programa obtendo a lista dos objetos carros, barcos e bicicletas. Para cada tipo, armazenamos os objetos em sua lista específica. Em seguida iteramos em cada lista e executamos o método de cada objeto.

Agora faremos a mesma coisa, mas com o uso de polimorfismo. O código 40 demonstra essa abordagem.

```
public static void main(String[] args) {

    List<Carro> carros = obterCarros();
    List<Barco> barcos = obterBarcos();
    List<Bicicleta> bicicletas = obterBicicletas();

    //Cria a lista de veículos (vazia)
    List<Veiculo> veiculos = new ArrayList<>();

    //Adiciona os objetos de cada tipo na lista genérica
    veiculos.addAll(carros);
    veiculos.addAll(barcos);
    veiculos.addAll(bicicletas);

    //Itera na lista de veiculo chamando o método
    for (Veiculo veiculo : veiculos) {
        veiculo.movimentar();
    }
}
```

Código 40 — Com uso de polimorfismo
Fonte: elaborado pelo autor (2024)

Inicialmente obtemos os objetos de cada tipo. Em seguida criamos uma lista vazia de tipo da superclasse Veículo. Depois, copiamos todos os objetos das listas da subclasse na lista da superclasse e, por fim, executamos o método “movimentar” de cada objeto obtido no início do programa.

Isso só é possível pois as classes Carro, Barco e Bicicleta são subclasses de Veículo. Caso contrário teríamos um erro de compilação.

Reforçando, o método “movimentar” executado é de cada objeto **da classe filha**, ou seja, o código executado é o da classe origem da instância.

No código 41, ajustamos o código anterior passando a responsabilidade de obter os objetos para o método em questão. Assim o programa ficou bem mais limpo.

```
public static void main(String[] args) {

    //Obtem a lista do tipo Veiculo
```



```
List<Veiculo> veiculos = obterVeiculos();

//Itera na lista de veículos,
//executando o método específico de cada objeto.
for (Veiculo veiculo : veiculos) {
    veiculo.movimentar();
}
}
```

Código 41 — Ajuste
Fonte: elaborado pelo autor (2024)

O mais interessante nesta abordagem é que o loop “for” executa o “movimentar” dos veículos sem precisar saber quem ou o que está sendo executado. Se novas subclasses de Veículo surgirem ou se alterarmos uma regra de uma subclasse existente, este loop será executado do mesmo modo sem precisar ser alterado.

Dessa forma atingimos o princípio “O - Open Closed” do SOLID, pois adicionamos novas funcionalidades no sistema sem precisar mudar (ou mudando pouco) outras partes do sistema.

Concluimos que polimorfismo é a possibilidade de que métodos de classes derivadas (subclasses) sejam chamados apropriadamente, em tempo de execução, com base no tipo real do objeto e não no tipo de sua referência ou variável.

O QUE VOCÊ VIU NESTA AULA?

Vimos nessa aula todos os conceitos de POO, assim como a diferença de não os usar. Vimos como o uso de POO deixa o código limpo e reutilizável, sendo possível aplicar conceitos de boas práticas como o SOLID. Nas videoaulas, aplicamos os conceitos explicados em um sistema real.

EMANDA

REFERÊNCIAS

CARDOSO, C. **Orientação a Objetos na Prática**. São Paulo: Ciência Moderna, 2020.

MARTIN, R. **Arquitetura Limpa**. Rio de Janeiro: Alta Books, 2018.

MARTIN, R. **Código Limpo**. Rio de Janeiro: Alta Books, 2011.

EXEMPLO

PALAVRAS-CHAVE

Herança. Polimorfismo. Classes e Objetos.

EMSE



POSTECH